

도착 시각(release time)과 soft due date를 갖는 컴파운드 트럭 크로스도킹 스케줄링: 정확해 검증을 동반한 병목 유도 Iterated Local Search

김영수* 권상진†

연세대학교 인공지능학과, 서울

Abstract

컴파운드 트럭(compound truck)이 부분 하역(partial unloading)된 뒤 한 목적지의 outbound carrier가 되는 다중 도어 크로스도킹의 트럭 스케줄링 문제를 다룬다. 기존 컴파운드 트럭 모델은 모든 트럭이 시각 0에 가용하다고 가정하고 makespan만 최소화한다. 본 연구는 트럭별 도착 시각(release time)과 soft due date를 도입하여 makespan과 총 tardiness의 가중합을 최소화하는 문제로 확장한다. 이를 mixed-integer program(MILP)과 CP-SAT 모형으로 정식화하고, 유효한 조합적 하한(lower bound)과 train/tuning/test seed를 분리한 재현 가능한 벤치마크를 제시한다. 제안 방법 VAA-GILS는 Vogel식 construction, best-improvement descent, 병목(bottleneck)을 겨냥한 guided 연산자, simulated annealing acceptance, kick restart를 결합한 iterated local search이다. CP-SAT 기준해가 존재하는 모든 셀에서 VAA-GILS는 그 incumbent 대비 0.1–0.6% 이내의 목적값을 1초 미만에 얻었으며(CP-SAT는 76–376초), 소형 무(無)시간창 인스턴스에서는 증명된 최적해와 일치하였다. CP-SAT가 600초 내 가능해를 내지 못하는 대형 시간창 셀에서는 비교 대상 중 best로 관측된 해를 제공하였다. 통제된 ablation은 성능이 descent, guided 연산자, restart라는 결정론적 구조에서 나오며, learned operator selection(tabular Q-learning, transfer DQN), 학습된 초기해, 확률적 acceptance는 이 환경에서 동일 예산의 uniform sampling을 능가하지 못함을 보인다.

주제어: 크로스도킹; 컴파운드 트럭; release time; soft due date; iterated local search; constraint programming; guided local search

1 서론

크로스도킹은 입고 화물을 창고에 보관하지 않고 곧바로 출고 트럭으로 환적하여 재고와 리드타임을 줄이는 물류 방식이다 [3, 13]. Joo와 Kim [4]이 도입하고 Shahmardan과 Sajadieh [12]가 발전시킨 컴파운드 트럭 변형에서는 한 대의 트럭이 두 역할을 겸한다: 여러 목적지 화물을 싣고 도착해 부분적으로 하역되고(한 목적지의 수요는 retain), 그 목적지의 나머지 수요를 적재한 뒤 carrier로 출발한다. 부분 하역은 전량 하역 대비 makespan을 크게 줄이는 것으로 보고되어 LTL 통합 네트워크에서 실용적 매력이크다.

기존 컴파운드 트럭 모델의 두 가정이 현실성을 제약한다. 첫째, 모든 트럭이 시각 0에 가용하다고 가정하지만 실제 도크는 시차를 둔 예약 도착을 마주한다. 둘째, 목적함수가 순수 makespan이지만 outbound 출발은 위반 시 비용이 발생하는 due date에 묶여 있다.

*제1저자.

†교신저자. Email: [지도교수 이메일 기입].

release time, time window, tardiness 자체는 일반 크로스도킹 스케줄링에서 이미 연구되어 왔다 [14, 1, 2, 10, 11, 9]. 그러나 컴파운드 트럭 설정은 temporal constraint의 의미를 바꾼다. 컴파운드 트럭은 순수 inbound도 outbound도 아니다: 부분 하역되어 한 목적지를 retain하고, 그 목적지로 들어오는 transfer들을 기다린 뒤 비로소 carrier로 출발한다. 이 부분 하역 변환(partial-unloading transformation)은 하나의 precedence chain — arrival → partial unload → retained cargo → transfer wait → outbound departure — 을 만들고, 이 chain은 트럭들을 공유 목적지를 통해 결합한다. 따라서 release time을 더하는 것은 한 트럭의 시작을 미루는 데 그치지 않는다: 늦은 도착은 그 트럭의 transfer에 의존하는 모든 carrier로 전파되고, soft due date는 이 연쇄 지연을 tardiness 비용으로 바꾼다. 본 연구의 기여는 이 상호작용 — 알려진 release time과 soft due date가 부분 하역 carrier 변환과 결합되는 방식 — 을 정식화하고 분석하는 것이며, 저자가 아는 한 이 결합은 이전에 모형화된 바 없다.

본 논문의 기여는 세 가지다. 첫째, 독립 schedule evaluator와 일치하는 big- M MILP 및 reified CP-SAT 정식화와, 목적함수에 대한 유효 조합 하한을 제시한다. 둘째, 기존 VAA construction과 generic neighborhood에 full best-improvement descent, 병목 유도 연산자, kick restart를 더한 VAA-GILS를 제안한다. 셋째, 통제된 ablation으로 성능이 search engine에서 오는지 learned operator selection에서 오는지 구분하며, 후자는 동일 예산의 uniform control을 증가하지 못함을 보인다. 이 결과는 강한 search engine 내부에서 learned selector를 평가할 때 동일 예산의 uniform control이 필요함을 시사한다.

2 관련 연구

시간 제약이 있는 크로스도킹 스케줄링. Boysen과 Fliedner [3]는 도크 스케줄링 문제를 분류하였고, Van Belle 등 [13]은 분야를 조망하였다. 여러 결정론적 모형이 conventional inbound/outbound 트럭에 대해 시간 제약을 이미 반영한다: Van Belle 등 [14]은 다중 도어·time window·tardiness를, Assadi와 Bagheri [1]는 ready time과 outbound earliness/tardiness를, Bodnar 등 [2]은 time window와 mixed-service door를 다룬다. Molavi 등 [10]은 hard due date를, Rijal 등 [11]은 truck scheduling과 door assignment의 통합을 다룬다. 이들은 release/due date가 새롭지 않음을 보이나, inbound/outbound 분리를 유지하며 부분 하역 후 carrier가 되는 트럭을 모형화하지 않는다.

불확실·불완전 도착 정보. 도착 불확실성은 별도의 흐름이다. Konur와 Golias는 bounded arrival window [5]와 unknown arrival 하의 cost-stable schedule [6]을 다룬다. Larbi 등 [8]은 도착 정보의 full/partial/absent를 구분하고, Ladier와 Alpan [7]은 time window 하의 robust schedule을, Xi 등 [17]은 two-stage conflict robust optimization을 제시한다. 본 모형은 결정론적이며 각 release time은 스케줄 구성 시점에 알려져 있어, 이들 불확실성 모형은 직접 대체가 아니라 자연스러운 확장에 해당한다.

최근집 이웃 문제와 학습형 메타휴리스틱. Li 등 [9]은 mixed service mode door와 time window의 통합 트럭 배정·스케줄링을 Q-learning 기반 ALNS로 푼다. 그 유연성은 도어 수준에 있고 트럭은 순수 inbound/outbound인 반면, 본 문제의 유연성은 트럭 수준(컴파운드 트럭, 도어 간 직접 transfer)에 있다. 한편 학습 기반 operator selection이 메타휴리스틱을 개선한다는 보고가 많으나, 본 연구는 강한 지역 탐색을 갖춘 뒤에는 learned selector의 이득이 작다는 통제된 반례를

더한다.

3 문제 정의

3.1 기본 모델

I, F, D, K, M 을 각각 컴파운드 트럭, outbound 트럭, 목적지, 상품 종류, 도어의 집합이라 하고 $|I| + |F| = |D|, |I| \leq |M|$ 이라 하자. 컴파운드 트럭 i 는 목적지 d 행 상품 k 를 f_{idk} 단위 싣는다. 단위 handling time t_k , 도어 m, n 간 배치 transfer time t_{mn} , 트럭 i 의 진입/출차 changeover time DE_i, DL_i 에 대해 $h_{id} = \sum_k f_{idk} t_k, H_i = \sum_d h_{id}$ 로 둔다. 각 컴파운드 트럭은 retain 목적지 하나와 도어 하나를(도어당 컴파운드 최대 1대), 각 outbound 트럭은 남은 목적지 하나와 도어를 고르며, 같은 도어의 outbound 트럭은 순서화된다. 각 목적지는 정확히 하나의 carrier를 갖는다.

타이밍은 evaluator 규칙을 따른다. 컴파운드 트럭 i (retain d)의 하역은 $r_i + DE_i + \sum_{d' \neq d} h_{id'}$ 에 끝난다. carrier는 기여하는 모든 source의 하역과 도어 간 transfer가 끝나야 loading할 수 있다. outbound carrier f 는 추가로 자신의 release와 도어의 선행 점유가 끝나는 시점에 door service를 시작하여 $S_f = \max(\text{door available, destination ready, } r_f)$ 이며, 이후 docking(DE_f), loading, undocking(DL_f)을 차례로 수행하므로 completion은 $S_f + DE_f + L_d + DL_f$ 이다. 즉 S_f 는 loading이 아니라 door service의 시작이고, loading은 $S_f + DE_f$ 에 시작한다. makespan $\tau = C_{\max}$ 는 최대 완료 시각이다.

3.2 Mixed-integer 정식화

이진 변수 x_{idm} 은 컴파운드 트럭 $i \in I$ 를 retain 목적지 d ·도어 m 에, z_{fdm} 은 outbound 트럭 $f \in F$ 를 배정한다. $X_{im} = \sum_d x_{idm}, Z_{fm} = \sum_d z_{fdm}, Z_{fd} = \sum_m z_{fdm}$ 로 두고, b_{fgm} 은 도어 m 에서 f 가 g 에 선행하면 1이다. 연속 변수 $U_i, C_i, S_f, C_f, C_{\max}$ 는 각각 컴파운드 하역 완료, 컴파운드 완료, outbound door-service 시작, outbound 완료, makespan이다. $L_{id} = \sum_{j \neq i} h_{jd}, L_d = \sum_i h_{id}, B$ 는 유효한 horizon 상한이다.

$$\sum_{d,m} x_{idm} = 1 \quad \forall i, \quad \sum_{d,m} z_{fdm} = 1 \quad \forall f, \quad (1)$$

$$\sum_{i,m} x_{idm} + \sum_{f,m} z_{fdm} = 1 \quad \forall d, \quad \sum_{i,d} x_{idm} \leq 1 \quad \forall m. \quad (2)$$

컴파운드 타이밍과 incoming-transfer precedence는

$$U_i = r_i + DE_i + H_i - \sum_{d,m} h_{id} x_{idm} \quad \forall i, \quad (3)$$

$$C_i \geq U_i + L_{id} + DL_i - B(1 - x_{idm}) \quad \forall i, d, m, \quad (4)$$

$$C_i \geq U_j + t_{nm} + L_{id} + DL_i - B(2 - x_{idm} - X_{jn}) \quad (5)$$

로, 모든 i, d, m, n 과 $\sum_k f_{jdk} > 0$ 인 기여 $j \neq i$ 에 대해 성립한다. outbound carrier 에 대해서는

$$S_f \geq r_f \quad \forall f, \quad (6)$$

$$C_f \geq S_f + DE_f + L_d + DL_f - B(1 - Z_{fd}) \quad \forall f, d, \quad (7)$$

$$S_f \geq U_i + t_{nm} - B(2 - z_{fdm} - X_{in}), \quad (8)$$

$$S_f \geq C_i - B(2 - Z_{fm} - X_{im}) \quad \forall f, i, m. \quad (9)$$

같은 도어 m 의 outbound 쌍 $f < g$ 에 대해 $b_{fgm} + b_{gfm} \geq Z_{fm} + Z_{gm} - 1, b_{fgm} \leq Z_{fm}, b_{gfm} \leq Z_{gm}$ 이며 non-overlap 은

$$S_g \geq C_f - B(1 - b_{fgm}), \quad S_f \geq C_g - B(1 - b_{gfm}). \quad (10)$$

끝으로 $C_{\max} \geq C_i \forall i, C_{\max} \geq C_f \forall f$. CP-SAT 모형은 big- M disjunction 을 enforcement literal 로 대체하고 시간을 0.01 단위로 정수화하며, 모든 배정을 고정하면 독립 evaluator 의 완료 시각이 재현되어 model fidelity 가 확인된다.

3.3 시간창 확장과 목적함수

각 트럭 $q \in I \cup F$ 는 release time $r_q \geq 0$ 과 soft due date $\bar{d}_q$ 를 갖고 $T_q \geq \max\{0, C_q - \bar{d}_q\}$ 이다. 목적함수는

$$\min J = C_{\max} + \lambda \sum_{q \in I \cup F} T_q, \quad \lambda = 1. \quad (11)$$

$r_q = 0, \bar{d}_q = \infty$ 이면 기존 makespan 모델로 환원된다. CP-SAT 는 incumbent 와 증명된 하한을 함께 반환하며, 하한과 일치가 증명될 때에만 최적으로 부른다.

3.4 조합적 하한

정확해 하한이 약한 셀의 gap 보고를 위해 도어 경합을 완화한 두 유효 하한을 사용한다. 컴파운드 트럭 i (retain d) 의 최속 완료와 outbound 트럭 f 의 최속 완료는

$$EF_i = r_i + DE_i + \min_{d \in D} \left(H_i - h_{id} + \sum_{j \in I \setminus \{i\}} h_{jd} \right) + DL_i, \quad (12)$$

$$EF_f = r_f + DE_f + \min_{d \in D} \sum_{j \in I} h_{jd} + DL_f. \quad (13)$$

critical-chain 하한은 $LB_{cc} = \max_{q \in I \cup F} EF_q$ 이다. door-area 하한은 각 트럭의 최소 도어 점유

$$o_i = DE_i + \left(H_i - \max_d h_{id} \right) + \min_d \sum_{j \in I \setminus \{i\}} h_{jd} + DL_i, \quad o_f = DE_f + \min_d \sum_{j \in I} h_{jd} + DL_f$$

을 도어 수로 나눈 $LB_{da} = \frac{1}{|M|} (\sum_i o_i + \sum_f o_f)$ 이다. 둘 다 제약을 완화하므로 $LB_\tau = \max\{LB_{cc}, LB_{da}\}$ 는 makespan 의 유효 하한이며, 각 EF_q 가 C_q 를 하방 바운드하므로 $LB_T = \sum_{q: \bar{d}_q < \infty} \max\{0, EF_q - \bar{d}_q\}$ 이고 $LB_J = LB_\tau + \lambda LB_T$ 는 식 (11) 의 유효 하한이다.

4 VAA-GILS

VAA-GILS는 iterated local search의 구현체로, 연산자 내부 sampling과 acceptance를 제외하면 결정론적이다.

Construction. VAA는 Vogel식 regret으로 retain 목적지를, 완료시간 우선순위로 outbound 목적지를 배정하고, finish time이 가장 이른 도어에 삽입한다. 모든 finish time 계산에 release time을 반영한다.

Descent. 세 무브군(outbound 재배치, 컴파운드 도어 스왑/빈 도어 이동, 두 트럭 목적지 스왑)에 대한 best-improvement 지역탐색으로, local optimum에서 종료하며 초기해·모든 신규 incumbent·최종해에 적용한다.

Iterated search. 1,000회 반복한다. selection policy가 연산자 하나를 뽑아 현재 해에서 후보를 생성하고, 후보가 전역 best를 개선하면 descent를 적용해 incumbent를 갱신한다. acceptance는 초기 온도 $T_0 = \max\{1, 0.05 J(s_0)\}$, 기하 냉각 $T \leftarrow 0.995 T$ 의 simulated annealing이다. 30회 무개선 시 best에서 random kick 3회로 재시작하고 $T \leftarrow \max\{T, 0.02 J(s^*)\}$ 로 reheat한다.

Operator pool. generic neighborhood 7종에, 현재 병목을 먼저 식별한 뒤 그 주변을 best-of로 탐색하는 guided 연산자 4종을 더한다: (g1) critical door의 마지막 outbound 트럭을 최선 도어·위치로 재배치, (g2) critical 트럭의 목적지를 전 트럭과 대조해 최선 스왑, (g3)/(g4) most tardy 트럭 기준의 동일 무브(tardy인 트럭이 없으면 g1/g2로 fallback). 고속 증분 evaluator(평가당 수십 μs)가 descent와 guided 무브를 저렴하게 만든다.

Selection policy와 원 모델 대비. selection policy는 plug-in이며 기본값은 uniform random이다. learned policy(SA-RL5의 tabular Q-learning, 규모 불변 피쳐로 오프라인 학습한 transfer DQN)는 ablation 목적으로만 평가하며 제안 방법의 일부가 아니다. 원 모델 SA-RL5는 RL이 단일 무브를 고르고 SA accept/reject로 스텝이 끝나는 순수 simulated annealing인 반면, VAA-GILS는 그 한 연산자 스텝을 iterated local search 안에 넣고 guided 연산자, 개선 무브마다의 full descent, kick restart를 더한다. 가장 중요한 차이는 개선 무브 이후의 descent이며, ablation은 이것이 품질 이득의 대부분을 설명함을 보인다. 표 1가 두 방법의 구조적 차이를 요약한다.

Table 1: 원 모델 SA-RL5와 VAA-GILS의 구조 비교.

항목	SA-RL5 (base)	VAA-GILS (제안)
전체 틀	순수 simulated annealing	iterated local search (SA는 acceptance만)
반복당 연산자	1개	1개 (동일)
연산자 pool	이웃 7종	그 7종 + guided 4종
연산자 선택	online Q-learning/roulette	uniform random (학습은 ablation 전용)
개선 무브 이후	없음 (SA step이 전부)	full best-improvement descent
정체 처리	없음	kick restart + reheat
acceptance	SA(Metropolis)	SA(Metropolis; ablation상 생략 가능)
목적함수	makespan	makespan + $\lambda \cdot$ tardiness

5 실험 설계

세 규모 ($|I|, |D|, |M|$): S (6, 9, 6), M (12, 18, 12), L (20, 30, 20), 컴파운드 비율 2/3을 사용한다. flow는 상품 3종에 대해 $[0, 20]$ 의 균등 정수, $t_k \sim U[1, 5]$, 도어는 100×100 랜덤 배치이다. 시간차 수준은 none/medium/tight이며 medium/tight에서 $r \sim U[0, \rho H]$, $\bar{d} = r + \delta H$, $(\rho, \delta) = (0.25, 0.60), (0.50, 0.35)$ 이다.

seed는 서로소인 train/tuning/test 풀로 분리한다. 주 해품질 비교(표 2)는 셀당 test 인스턴스 20개 $\times$ replication 5회를 사용하고, 통제된 ablation은 동일예산·동일구조 분해이므로 원래의 5-인스턴스 설계를 유지한다. 인스턴스별로 replication을 평균한 뒤 짝지은 인스턴스 평균에 양측 Wilcoxon signed-rank test를 적용한다. GILS는 1,000회 반복한다. CP-SAT는 인스턴스당 1회, 8 thread로 S 300초(5개), M/L 600초(셀당 2개) 실행하며 S-none 5개만 최적이 증명된다. 주 지표는 전 방법·전 예산·CP-SAT를 포함해 best로 관측된 해에 대한 인스턴스별 gap Δ_{bo} 이다.

6 실험 결과

6.1 해 품질

표 2이 20-인스턴스 test 결과를 요약한다. 평균목적값· Δ_{bo} 는 20-인스턴스 기준이고, vs CP-SAT는 CP-SAT 기준해가 있는 부분집합(S 셀 5개, M-none 2개)만이며 S-none만 최적이 증명된다. 정확해 기준이 있는 모든 셀에서 GILS는 그와 대등하다(vs CP-SAT 0.1–0.6% 이내). 기본(default) selector인 uniform 기준으로, GILS는 VAA를 평균 2.65%($n = 180, p < 10^{-4}$), SA-RL5를 1.38%($n = 60, p < 10^{-4}$) 능가한다. GILS의 평균 실행 시간은 S/M/L에서 0.11, 0.52, 2.3초, CP-SAT는 76, 237, 376초이다.

6.2 성능은 어디에서 오는가

세 가지 통제된 ablation(예산 1,000회 고정)으로 엔진을 분해한다.

Operator pool. selection을 uniform으로 고정하고 pool만 generic(7)/critical(+g1,g2)/full(+g3,g4)로 바꾸면 Δ_{bo} 가 9개 셀 전부에서 generic $\geq$ critical $\geq$ full로 단조 감소한다(그림 1). 짝지은 Wilcoxon(표 3)은 유도 연산자가 목적값을 일관되게·유의하게 낮춤을 보인다.

Table 3: Operator-pool ablation(양측 Wilcoxon; + 평균 = 풍부한 pool이 우세).

대비	n	개선(%p)	p
generic $\rightarrow$ critical (g1,g2 추가)	136	+0.114	$< 10^{-4}$
critical $\rightarrow$ full (g3,g4 추가)	172	+0.048	$< 10^{-4}$
generic $\rightarrow$ full (전체 유도)	166	+0.161	$< 10^{-4}$

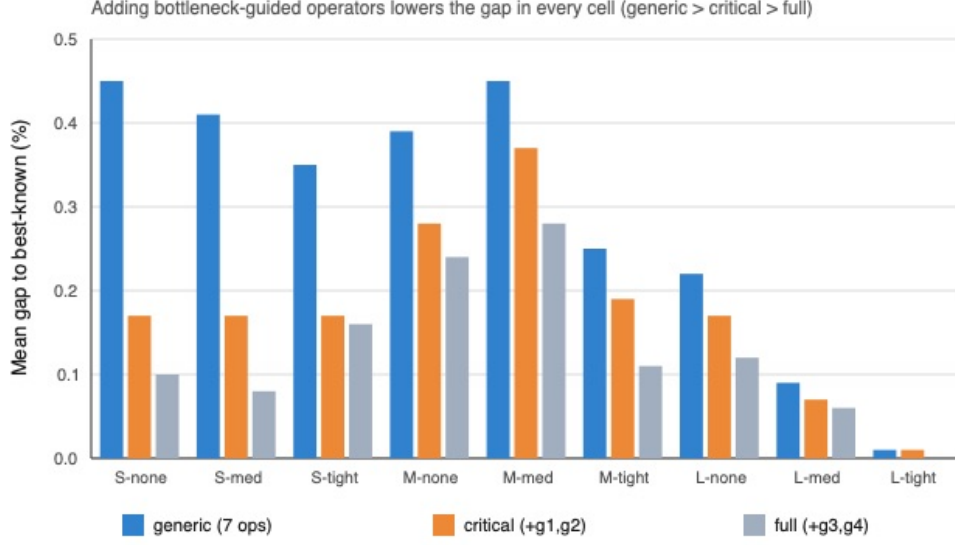


Figure 1: operator pool 별 best로 관측된 해 대비 gap.

Engine components. pool을 full, selection을 uniform으로 고정하고 컴포넌트를 하나씩 제거 하면 (표 4), best-improvement descent가 최대 기여자이고 kick restart가 그다음이며, VAA 초기해는 통계적으로 유의하지 않고 확률적 acceptance는 제거해도 목적값이 나빠지지 않는다(그림 2).

Table 4: Engine-component ablation(leave-one-out; 제거 시 상대 목적값 악화율).

제거 컴포넌트	n	악화(%p)	p
best-improvement descent	45	+0.156	$< 10^{-4}$
kick restart	43	+0.069	$< 10^{-4}$
VAA initial solution	45	+0.022	0.114
stochastic acceptance	45	-0.026	0.0014

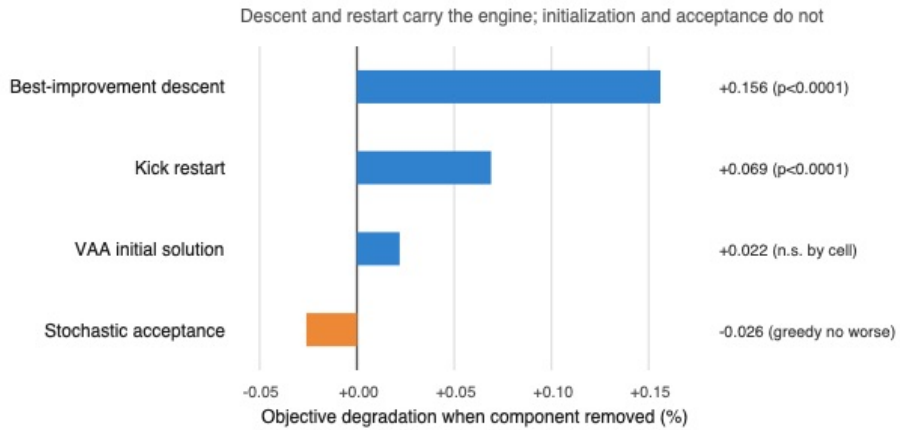


Figure 2: 컴포넌트 제거 시 목적값 악화율.

Learned operator selection. pool과 컴포넌트를 full로 고정하고 selection만 바꾸면 (표 5), 어떤 learned policy도 uniform을 실용적으로 능가하지 못한다. 예산 50-3,000회에서 uniform은 $0.47 \rightarrow 0.16\%$, tabular는 $0.56 \rightarrow 0.09\%$, DQN은 $0.56 \rightarrow 0.26\%$ 로 움직이며, 방향은 예산에 따라

뒤집힌다(그림 3). 이 negative result는 무시간창 셀만으로도 성립하므로 시간창 확장의 산물이 아니다.

Table 5: 1,000회에서 selection 효과(양측 Wilcoxon; 양의 평균 = 앞 방법 우세).

비교	평균차(%p)	p	판정
tabular vs uniform	-0.01	0.150	차이 없음
tabular vs DQN	+0.10	$< 10^{-4}$	DQN 열세
uniform vs DQN	+0.11	$< 10^{-4}$	DQN 열세

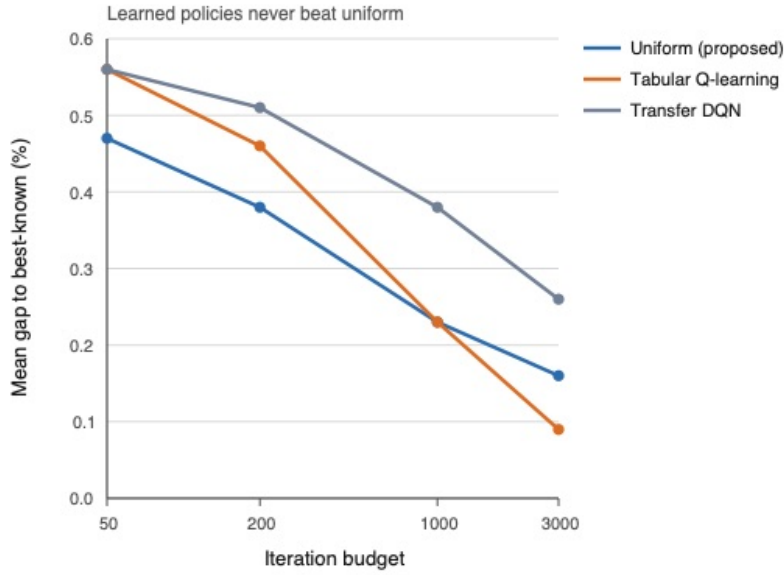


Figure 3: iteration budget에 따른 best로 관측된 해 대비 gap.

6.3 Tardiness-weight 민감도

식 (11)이 시간 단위 성분 둘을 합치므로 tuning 풀에서 $\lambda \in \{0, 0.25, 0.5, 1, 2, 4, 8\}$ 을 스윙한다. 물리적 makespan과 총 tardiness를 인스턴스별로 min-max 정규화해 평균한다(그림 4). tardiness 감소의 대부분은 $\lambda = 0$ 에서 1 사이에서 일어나고 이후에는 그 폭이 줄어든다. 전 스윙에서 평균 makespan은 약 1% 상승, 평균 tardiness는 2-3% 하락한다. $\lambda = 1$ 을 채택한다: 완만한 makespan 비용으로 tardiness가 거의 최소에 이르고 두 시간 성분에 동일 가중을 준다. 이 스윙은 test가 아닌 tuning 인스턴스를 사용한 설계 정당화이다.

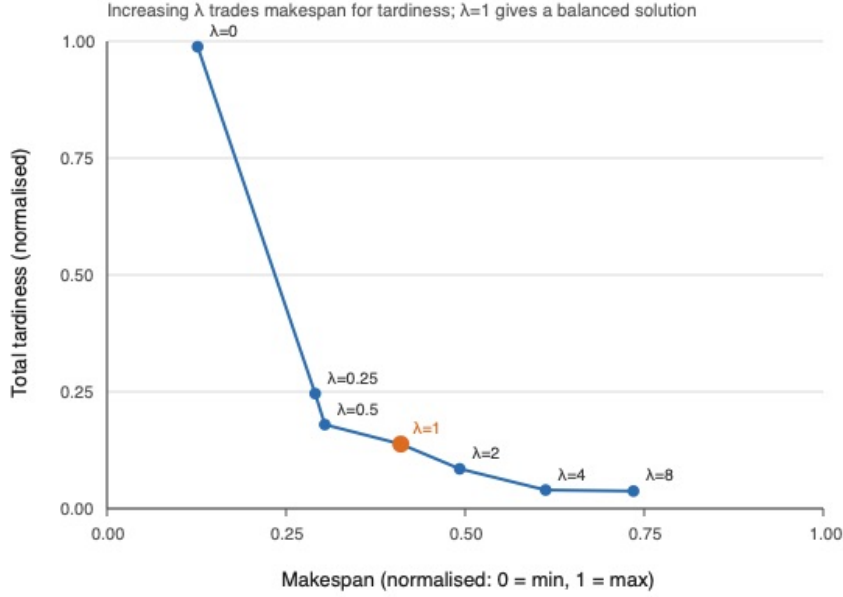


Figure 4: tuning 인스턴스에서의 정규화된 makespan-tardiness trade-off.

7 논의

selector 결과는 본 탐색 과정 안에서 해석해야 한다. 후보 평가가 수십 μs 로 저렴하고, 모든 신규 incumbent 뒤에 결정론적 descent가 따른다. 같은 basin에 들어간 두 실행은 선행 무브가 달라도 같은 해로 수렴할 수 있으며, budget 곡선은 이 수렴이 대부분의 test 인스턴스에서 1,000회 이전에 일어남을 시사한다. 같은 이유로 SA acceptance와 reheating은 기저 SA-RL5 framework와의 comparability를 위해 유지하였으나, ablation은 본 설정에서 이들이 해 품질에 필수적이지 않음을 보인다.

정책이 예측할 정보도 적다. 인스턴스는 정적이고 모든 release는 최적화 전에 알려져 있다. 이는 도착이 시간에 따라 드러나는 online dock control이나 한 무브 평가가 비싼 simulation 모형과 다르며, 그런 환경에서는 learned selection의 역할이 커질 수 있다. 본 결과가 뒷받침하는 것은 하나의 제한적 권고다: learned selector는 동일 엔진·동일 예산에서 uniform sampling과 비교되어야 하며, 그 통제가 없으면 descent·restart·강한 무브군의 이득이 정책 자체의 것으로 잘못 귀속될 수 있다.

8 결론

본 연구는 부분 하역 컴파운드 트럭 스케줄링에 release time과 soft due date를 도입하고, MILP·CP-SAT 정식화와 유효 하한으로 휴리스틱 해를 검증하였다. 정확해 기준이 있는 곳에서 VAA-GILS는 그 incumbent 대비 1% 미만이었고(소형은 증명된 최적), 대형 시간창 셀에서는 수 초 안에 비교 대상 중 best로 관측된 해를 제공하였다. ablation은 성능이 best-improvement descent, guided 무브, restart라는 결정론적 구조에서 나오며, learned operator selection·학습된 초기해·확률적 acceptance는 이 환경에서 실용적 이득을 주지 않음을 보였다. 향후 연구로 미공개(online) 도착의 동적 변형, 대형 시간창 셀의 더 tight한 하한, 산업 데이터 사례 연구를 제시한다.

A Transfer DQN 구현 세부

transfer DQN은 매 반복 11개 연산자 pool에서 하나를 고른다. Q-network는 27차원 규모 불변 상태를 연산자별 Q값으로 사상하는 MLP로, 64 unit의 hidden layer 2개와 ReLU를 쓴다. 학습은 용량 100,000의 replay buffer를 사용하며, 500 transition warm-up 후 매 step 1회 gradient step으로 Huber(smooth- L_1) temporal-difference loss를 Adam (learning rate 10^{-3} , discount $\gamma = 0.9$)으로 최소화한다. mini-batch는 64, target network는 500 step마다 동기화한다. reward는 shaped signal(신규 incumbent 2, 비악화 1, 그 외 0)이다. agent는 *train* 풀(S/M 규모, 전 flow·시간창)에서 500회 실행으로 오프라인 학습(ϵ -greedy) 후, *test* 인스턴스에 zero-shot ($\epsilon = 0$, 갱신 없음)으로 적용한다. 결과가 uniform 대비 개선이 없다는 것이므로, 재현·강화가 가능하도록 network·학습 스크립트·seed를 공개한다.

생성형 AI 사용 명시

원고 준비 과정에서 저자들은 언어 교정, 원고 구성, 참고문헌 확인, 계산 코드 개발·검토 보조를 위해 OpenAI Codex(GPT-5)와 Claude Code(Anthropic)를 사용하였다. 저자들은 모든 참고문헌·정식화·코드·결과·해석을 검토·검증하였으며 내용에 대한 전적인 책임을 진다.

References

- [1] M. T. Assadi and M. Bagheri, “Differential evolution and population-based simulated annealing for truck scheduling,” *Computers & Industrial Engineering*, vol. 96, pp. 149–161, 2016.
- [2] P. Bodnar, R. de Koster, and K. Azadeh, “Scheduling trucks in a cross-dock with mixed service mode dock doors,” *Transportation Science*, vol. 51, no. 1, pp. 112–131, 2017.
- [3] N. Boysen and M. Flidner, “Cross dock scheduling: Classification, literature review and research agenda,” *Omega*, vol. 38, pp. 413–422, 2010.
- [4] C. M. Joo and B. S. Kim, “Scheduling compound trucks in multi-door cross-docking terminals,” *International Journal of Advanced Manufacturing Technology*, vol. 64, pp. 977–988, 2013.
- [5] D. Konur and M. M. Goliass, “Analysis of different approaches to cross-dock truck scheduling with truck arrival time uncertainty,” *Computers & Industrial Engineering*, vol. 65, no. 4, pp. 663–672, 2013.
- [6] D. Konur and M. M. Goliass, “Cost-stable truck scheduling at a cross-dock facility with unknown truck arrivals,” *Transportation Research Part E*, vol. 49, no. 1, pp. 71–91, 2013.
- [7] A.-L. Ladier and G. Alpan, “Robust cross-dock scheduling with time windows,” *Computers & Industrial Engineering*, vol. 99, pp. 16–28, 2016.
- [8] R. Larbi, G. Alpan, P. Baptiste, and B. Penz, “Scheduling cross docking operations under full, partial and no information on inbound arrivals,” *Computers & Operations Research*, vol. 38, no. 6, pp. 889–900, 2011.

- [9] Y. Li, M. Mohammadi, X. Zhang, Y. Lan, and W. van Jaarsveld, “Integrated trucks assignment and scheduling with mixed service mode docks,” *European Journal of Operational Research*, vol. 333, no. 1, pp. 117–137, 2026.
- [10] D. Molavi, A. Shahmardan, and M. S. Sajadieh, “Truck scheduling in a cross docking systems with fixed due dates and shipment sorting,” *Computers & Industrial Engineering*, vol. 117, pp. 29–40, 2018.
- [11] A. Rijal, M. Bijvank, and R. de Koster, “Integrated scheduling and assignment of trucks at unit-load cross-dock terminals,” *European Journal of Operational Research*, vol. 278, no. 3, pp. 752–771, 2019.
- [12] A. Shahmardan and M. S. Sajadieh, “Truck scheduling in a multi-door cross-docking center with partial unloading,” *Computers & Industrial Engineering*, vol. 139, Art. 106134, 2020.
- [13] J. Van Belle, P. Valckenaers, and D. Cattrysse, “Cross-docking: State of the art,” *Omega*, vol. 40, no. 6, pp. 827–846, 2012.
- [14] J. Van Belle, P. Valckenaers, G. Vanden Berghe, and D. Cattrysse, “A tabu search approach to the truck scheduling problem with multiple docks and time windows,” *Computers & Industrial Engineering*, vol. 66, no. 4, pp. 818–826, 2013.
- [15] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [16] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [17] X. Xi, C. Liu, Y. Wang, and L. H. Lee, “Two-stage conflict robust optimization models for cross-dock truck scheduling under uncertainty,” *Transportation Research Part E*, vol. 144, Art. 102123, 2020.

Table 2: 20-인스턴스 test 결과. Δ_{bo} 는 best로 관측된 해 대비 평균 gap. vs CP-SAT는 CP-SAT 기준해가 있는 부분집합(S-none은 증명된 최적).

셀	방법	평균목적값 $\pm$ 표준편차	$\Delta_{bo}(\%)$	vs CP-SAT(%)
S-none	VAA	1497.5 ± 400.3	6.60	+6.49
	Paper-SA-RL5	1423.3 ± 383.6	1.03	+1.21
	GILS-uniform	1411.8 ± 382.7	0.16	+0.21
	GILS-tabular	1412.9 ± 381.3	0.28	+0.33
	GILS-DQN	1416.1 ± 382.4	0.51	+0.75
S-medium	VAA	5447.5 ± 1468.9	3.89	+3.33
	GILS-uniform	5255.7 ± 1413.3	0.08	+0.11
	GILS-tabular	5259.1 ± 1412.3	0.15	+0.18
	GILS-DQN	5270.8 ± 1415.3	0.38	+0.55
S-tight	VAA	9276.8 ± 2387.6	2.90	+2.17
	GILS-uniform	9010.8 ± 2206.1	0.09	+0.21
	GILS-tabular	9011.9 ± 2206.7	0.11	+0.17
	GILS-DQN	9022.0 ± 2205.3	0.23	+0.34
M-none	VAA	3030.9 ± 903.6	3.73	+4.80
	Paper-SA-RL5	2976.4 ± 864.4	1.86	+0.99
	GILS-uniform	2928.9 ± 842.6	0.29	+0.13
	GILS-tabular	2932.1 ± 846.3	0.37	+0.11
	GILS-DQN	2932.5 ± 845.5	0.40	+0.19
M-medium	VAA	23025.5 ± 4075.4	2.56	—
	GILS-uniform	22526.4 ± 3960.5	0.29	—
	GILS-tabular	22515.5 ± 3964.0	0.24	—
	GILS-DQN	22540.3 ± 3956.4	0.36	—
M-tight	VAA	38903.7 ± 11283.5	1.25	—
	GILS-uniform	38528.0 ± 11078.0	0.19	—
	GILS-tabular	38512.1 ± 11059.5	0.16	—
	GILS-DQN	38543.6 ± 11077.6	0.24	—
L-none	VAA	5492.3 ± 1482.2	2.21	—
	Paper-SA-RL5	5479.7 ± 1449.5	1.97	—
	GILS-uniform	5388.4 ± 1427.3	0.27	—
	GILS-tabular	5385.5 ± 1424.6	0.22	—
	GILS-DQN	5386.1 ± 1425.2	0.23	—
L-medium	VAA	55611.1 ± 18087.3	1.52	—
	GILS-uniform	54846.4 ± 17523.5	0.10	—
	GILS-tabular	54833.4 ± 17526.8	0.07	—
	GILS-DQN	54848.8 ± 17522.9	0.11	—
L-tight	VAA	119671.9 ± 32475.3	0.77	—
	GILS-uniform	118901.4 ± 31781.1	0.08	—
	GILS-tabular	118865.6 ± 31788.4	0.05	—
	GILS-DQN	118888.8 ± 31789.1	0.07	—